



Процессы в Linux

Программы, процессы, демоны, зомби, сироты



Проверить, идет ли запись

Напишите «+» в чат, если меня слышно и видно





Преподаватель



Андрей Буранов

Системный администратор в компании VK, департамент VKPlay.

Работаю с операционной системой Linux более 10 лет.

Преподаю на курсах Linux более 8 лет.

Топ-3 лучших преподавателей образовательных порталов.

- преподаватель и руководитель курсов

Administrator Linux.Basic

Administrator Linux. Professional

Administrator Linux. Advanced

Правила вебинара



Активно
участвуем



Задаем вопрос
в чат



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Пишем в чат



Документ



Ответьте себе или
задайте вопрос

Цели вебинара

После занятия вы сможете



1. Понимать отличие программы от процесса
2. Создавать демонов
3. Убивать зомби
4. Анализировать некоторые системные вызовы



Программа

- термин, в переводе означающий «предписание»
- комбинация компьютерных инструкций и данных
- обычно хранится в энергонезависимой памяти

Процесс

- термин, в переводе означающий «течение, ход, продвижение»
- выполнение пассивных инструкций компьютерной программы
- выполняющаяся программа и все её элементы (адресное пространство, глобальные переменные, регистры...)

Отличия

- программа — пассивная последовательность инструкций
- процесс — непосредственное выполнение этих инструкций

Программа



Процесс



Системные вызовы

- обращение прикладной программы к ядру операционной системы
- управляют процессами
- работают с файлами
- управляют устройствами
- работают с информацией
- обеспечивают связь и коммуникацию

Системные вызовы для управления процессами

`fork()` - создание процесса

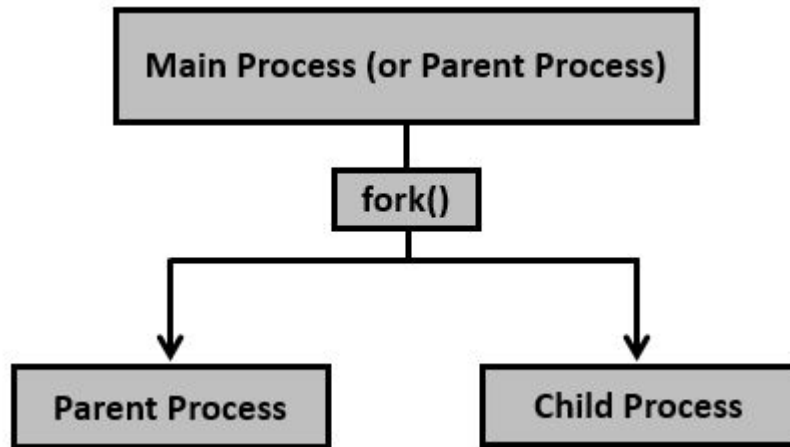
`exec()` - загрузка (запуск) программы в процесс

`exit()` - завершение процесса

`wait()` - считывание кода возврата процесса

fork() - единственный способ создать процесс в Linux

- стартует в одном процессе, завершается в двух
- процессу родителю возвращает код возврата равный PID дочернего процесса
- дочернему процессу возвращает код 0



exec() - способ запустить программу в Linux

- работает в существующем процессе
- заменяет программу процесса на ту, что была передана exec() в кач-ве аргумента
- exec() - семейство системных вызовов (exec, execl, execlp, execv, execve, execvp)

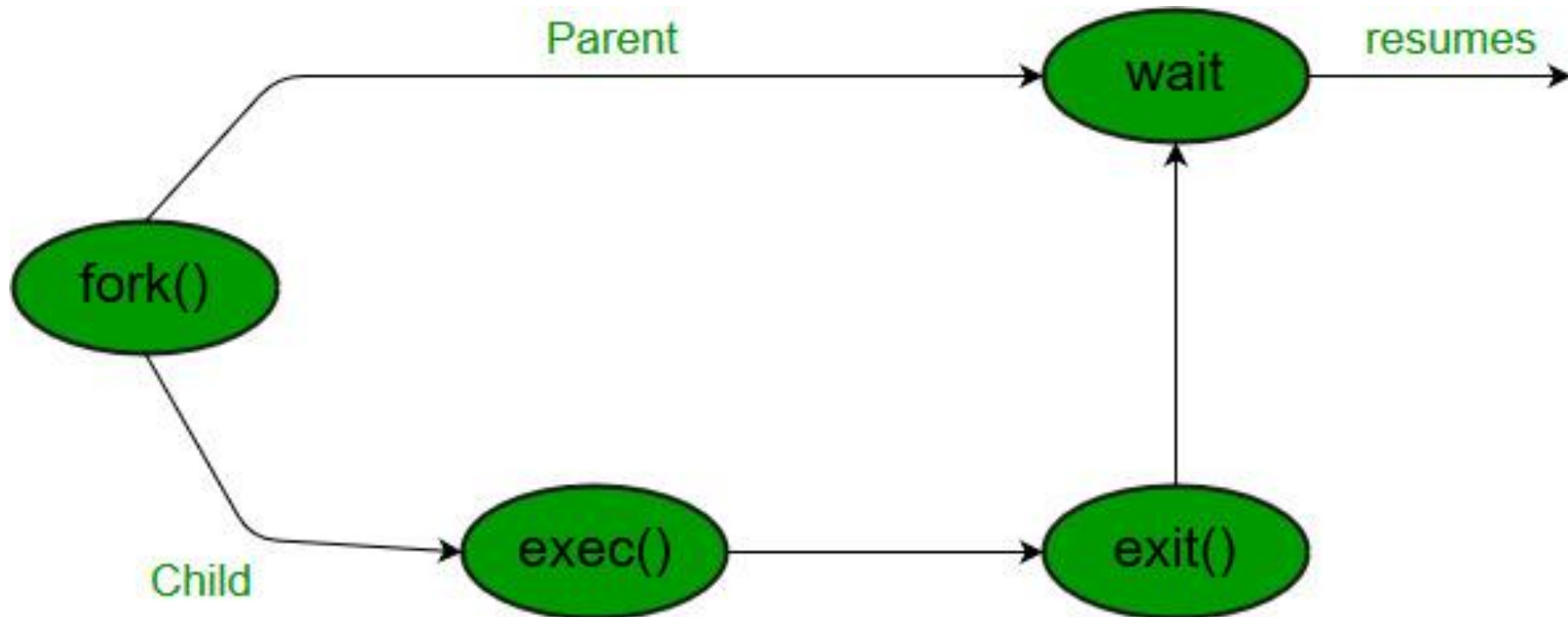
exit() - завершение работы программы в Linux

- приводит к обычному завершению программы
- после завершения программы ее переменная `status` будет доступна родительскому процессу (`status` - код возврата)
- `exit()` - не завершает процесс

wait() – считывает код возврата дочернего процесса в Linux

- приостанавливает выполнение текущего процесса до тех пор, пока дочерний процесс не прекратит выполнение
- получает код возврата дочернего процесса
- после вызова wait() дочерний процесс полностью исчезает из системы

Жизненный цикл процесса в Linux



Некоторые состояния процессов

R - выполняется в данный момент

S - процесс ожидает

D - ожидает ввода-вывода

T - процесс остановлен

Z - процесс зомби

Демон

- процесс работающий в фоновом режиме без прямого
- взаимодействия с пользователем
- sshd, httpd, ftpd и т.д. - программы работающие в режиме демона (последняя буква в названии "d" означает daemon)
- название произошло в честь демона Максвелла



Зомби

- обязательное состояние любого процесса
- процессы зомби не потребляют ресурсы
- процесс зомби нельзя убить (завершить)



Зомби

exit()



wait()



Сирота

- процесс оставшийся без родителя
- будет мгновенно усыновлен процессом init (в современных системах systemd)
- явление очень кратковременное (увидеть процесс сироту не получится)



Как убить зомби?

- А нужно ли...?

- сделать его сиротой
- заставить родителя послать `wait()`

Вопросы?



Задаем
вопросы в чат



Ставим “-”,
если вопросов нет

Тест

Вопросы для проверки

Что происходит при успешном вызове `fork()`?

- A. Возвращается 0 обоим процессам
- B. Возвращается PID родительского процесса обоим процессам
- C. Родитель получает PID дочернего процесса, а дочерний - 0
- D. Родитель получает 0, а дочерний PID родителя



Отправьте в чат верную букву

Вопросы для проверки

Что происходит при успешном вызове `fork()`?

- A. Возвращается 0 обоим процессам
- B. Возвращается PID родительского процесса обоим процессам
- C. Родитель получает PID дочернего процесса, а дочерний - 0**
- D. Родитель получает 0, а дочерний PID родителя

Вопросы для проверки

Какое из утверждений о `exec()` верное?

- A. Создаёт новый процесс и запускает указанную программу
- B. Заменяет текущий процесс на новую программу
- C. Загружает программу в память, но не запускает
- D. Может быть вызван только из ядра



Отправьте в чат верную букву

Вопросы для проверки

Какое из утверждений о `exec()` верное?

A. Создаёт новый процесс и запускает указанную программу

B. Заменяет текущий процесс на новую программу

C. Загружает программу в память, но не запускает

D. Может быть вызван только из ядра

Вопросы для проверки

Что произойдёт, если родительский процесс не вызовет `wait()` после завершения дочернего?

- A. Дочерний процесс станет зомби
- B. Дочерний процесс будет удалён автоматически
- C. Родитель будет завершён сигналом SIGCHLD
- D. Дочерний процесс станет демоном



Отправьте в чат верную букву

Вопросы для проверки

Что произойдёт, если родительский процесс не вызовет `wait()` после завершения дочернего?

A. Дочерний процесс станет зомби

B. Дочерний процесс будет удалён автоматически

C. Родитель будет завершён сигналом SIGCHLD

D. Дочерний процесс станет демоном

Вопросы для проверки

Какой параметр `strace` позволяет подключиться к уже запущенному процессу по его PID?

- A. -h
- B. -o
- C. -T
- D. -p



Отправьте в чат верную букву

Вопросы для проверки

Какой параметр `strace` позволяет подключиться к уже запущенному процессу по его PID?

- A. `-h`
- B. `-o`
- C. `-T`
- D. `-p`

Вопросы для проверки

Для какой из перечисленных задач наиболее корректно использовать `strace`?

- A. Отслеживать вызовы функций стандартной библиотеки программы
- B. Наблюдать за системными вызовами и сигналами между процессом и ядром
- C. Измерять время выполнения отдельных функций программы
- D. Проверять работу библиотек при работе с сетью



Отправьте в чат верную букву

Вопросы для проверки

Для какой из перечисленных задач наиболее корректно использовать `strace`?

A. Отслеживать вызовы функций стандартной библиотеки программы

B. Наблюдать за системными вызовами и сигналами между процессом и ядром

C. Измерять время выполнения отдельных функций программы

D. Проверять работу библиотек при работе с сетью

Задания на выбор:

1. написать свою реализацию ps ах используя анализ /proc
Результат ДЗ - рабочий скрипт который можно запустить
2. написать свою реализацию lsof
Результат ДЗ - рабочий скрипт который можно запустить
3. дописать обработчики сигналов в прилагаемом скрипте, протестировать, приложить сам скрипт, инструкции по использованию
Результат ДЗ - рабочий скрипт который можно запустить + инструкция по использованию и лог консоли
4. реализовать 2 конкурирующих процесса по IO. пробовать запустить с разными iocb
Результат ДЗ - скрипт запускающий 2 процесса с разными iocb, измеряющий время выполнения и лог консоли
5. реализовать 2 конкурирующих процесса по CPU. пробовать запустить с разными nice
- Результат ДЗ - скрипт запускающий 2 процесса с разными nice и измеряющий время выполнения и лог консоли

Итоги вебинара



Проверка достижения целей

1. Понимать отличие программы от процесса
2. Создавать демонов
3. Убивать зомби
4. Анализировать некоторые системные вызовы

Заполните, пожалуйста,
опрос о занятии

Спасибо за внимание!